

ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



MÔN HỌC: PHÂN TÍCH THIẾT KẾ PHẦN MỀM (N05)
ĐỀ TÀI: “NUTRISCAN MOBILE – HEALTRACK DAILY”

Thành viên nhóm

Họ và tên	Mã sinh viên	Lớp
Trịnh Gia Huy	23010069	K17-KHMT(AI&KHDL)_1
Bùi Trung Kiên	23010380	K17-KHMT(AI&KHDL)_1
Nguyễn Đình Đức Trung	22010052	K16-KHMT(AI&KHDL)_1

NHÓM 9

GIÁO VIÊN GIẢNG DẠY: ThS Vũ Quang Dũng

Hà Nội, tháng 6 năm 2026

ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



MÔN HỌC: PHÂN TÍCH THIẾT KẾ PHẦN MỀM (N05)

ĐỀ TÀI:

“NUTRISCAN MOBILE – HEALTRACK DAILY”

Thành viên nhóm

Họ và tên	Mã sinh viên	Phân công công việc	Đóng góp
Trịnh Gia Huy	23010069	Phụ trách AI, logic đề xuất	100%
Bùi Trung Kiên	23010380	Xây dựng OCR, DB	100%
Nguyễn Đình Đức Trung	22010052	Xây dựng UI/UX	100%

Giáo viên hướng dẫn: ThS Vũ Quang Dũng

Hà Nội, tháng 6 năm 2026

Mục lục

1	Phân tích Yêu cầu (Software Requirements Specification)	1
1.1	Giới thiệu tổng quan	1
1.1.1	Mục đích tài liệu	1
1.1.2	Phạm vi hệ thống	1
1.1.3	Định nghĩa, viết tắt và thuật ngữ	2
1.2	Danh sách Actor	2
1.3	Danh sách Use Case	3
1.4	Yêu cầu chức năng (Functional Requirements)	5
1.4.1	Nhóm chức năng Xác thực	5
1.4.2	Nhóm chức năng Dinh dưỡng	5
1.4.3	Nhóm chức năng Quét	6
1.4.4	Nhóm chức năng Luyện tập	6
1.5	Yêu cầu phi chức năng	6
2	Mô hình Use Case	8
2.1	Biểu đồ Use Case tổng thể	8
2.2	Kịch bản Use Case chính	9
2.2.1	UC-07: Quét và Phân tích Món ăn	9
2.2.2	UC-12: Lên lịch Luyện tập thông minh	10
2.2.3	UC-06: Thêm bữa ăn vào nhật ký	10
2.2.4	UC-10: Nhận đề xuất thực phẩm thông minh	11
3	Thiết kế Hướng đối tượng – Class Diagram	12
3.1	Tổng quan kiến trúc lớp	12
3.2	Class Diagram – Module Xác thực (Auth)	13
3.3	Class Diagram – Module Dinh dưỡng (Nutrition)	14

3.4	Class Diagram – Module Luyện tập (Workout)	15
4	Thiết kế Tương tác – Sequence Diagram	16
4.1	Sequence Diagram: Đăng nhập (UC-02)	16
4.2	Sequence Diagram: Thêm bữa ăn (UC-06)	17
4.3	Sequence Diagram: Quét Món ăn (UC-07)	18
4.4	Sequence Diagram: Lên lịch Luyện tập (UC-12)	19
5	Thiết kế Hành vi – State Machine & Activity Diagram	20
5.1	State Machine – AuthCubit	20
5.2	State Machine – Quy trình Quét Sản phẩm	21
5.3	Activity Diagram – Luồng Onboarding	22
5.4	Activity Diagram – Luồng Thêm bữa ăn	23
6	Kiến trúc Hệ thống	24
6.1	Tổng quan kiến trúc	24
6.2	Package Diagram	25
6.3	Kiến trúc phân tầng	26
6.4	Các quyết định thiết kế quan trọng	26
7	Design Patterns	28
7.1	Pattern 1: Repository Pattern	28
7.1.1	Vấn đề	28
7.1.2	Pattern được chọn: Repository Pattern	28
7.1.3	Cài đặt (Code)	29
7.2	Pattern 2: Cubit / BLoC Pattern (State Management) . .	31
7.3	Pattern 3: Strategy Pattern (Recommendation Engine) . .	32
8	Cài đặt và Giao diện	34
8.1	Môi trường phát triển	34
8.2	Hướng dẫn chạy ứng dụng	34
8.3	Mô tả giao diện các chức năng chính	35
8.3.1	Màn hình Đăng nhập / Đăng ký	35
8.3.2	Màn hình Onboarding	36
8.3.3	Dashboard – Trang chủ	36

8.3.4	Foods/Discover	37
8.3.5	Quét (Scan)	38
8.3.6	Workout	39
9	Kiểm thử	41
9.1	Chiến lược kiểm thử	41
9.2	Bảng Test Cases	41
9.3	Unit Test – Ví dụ cài đặt	43
9.4	Báo cáo kết quả kiểm thử	45
10	Kết luận	46
10.1	Kết quả đạt được	46
10.1.1	Về mặt kỹ thuật	46
10.1.2	Về mặt chức năng	46
10.2	Khó khăn gặp phải	47
10.3	Hướng phát triển	47
A	Phụ lục	48
A.1	Biên bản họp nhóm	48
A.1.1	Biên bản họp tuần 1	48
A.1.2	Biên bản họp tuần 2	48
A.1.3	Biên bản họp tuần 3	49
A.1.4	Biên bản họp tuần 4	49
A.1.5	Biên bản họp tuần 5	50
A.1.6	Biên bản họp tuần 6	50
A.2	Liên kết Repository	50
A.3	Schema Cơ sở dữ liệu SQLite	51
A.4	Bảng so sánh với các ứng dụng tương tự	53

Danh mục hình ảnh

2.1	Biểu đồ Use Case tổng thể – NutriScan Mobile	8
3.1	Class Diagram – Module Xác thực (Auth)	13
3.2	Class Diagram – Module Dinh dưỡng (Nutrition)	14
3.3	Class Diagram – Module Luyện tập (Workout)	15
4.1	Sequence Diagram – Đăng nhập (UC-02)	16
4.2	Sequence Diagram – Thêm bữa ăn (UC-06)	17
4.3	Sequence Diagram – Quét món ăn (UC-07)	18
4.4	Sequence Diagram – Lên lịch luyện tập (UC-12)	19
5.1	State Machine Diagram – Trạng thái xác thực (AuthCubit)	20
5.2	State Machine Diagram – Quy trình quét sản phẩm	21
5.3	Activity Diagram – Luồng Onboarding và khởi động	22
5.4	Activity Diagram – Luồng thêm bữa ăn	23
6.1	Package Diagram – Tổ chức gói (Package Organization)	25
6.2	Kiến trúc phân tầng – Clean Architecture	26
7.1	Repository Pattern – Minh họa NutritionRepository	29
7.2	Cubit Pattern – Luồng quản lý trạng thái dinh dưỡng	31
8.1	Giao diện xác thực người dùng của NutriScan Mobile	36
8.2	Nhóm màn hình theo dõi sức khỏe và hồ sơ cá nhân	37
8.3	Giao diện gợi ý thực phẩm dựa trên nhu cầu dinh dưỡng	38
8.4	Giao diện quét món ăn bằng camera	39
8.5	Giao diện lên lịch bài tập thông minh	40

Danh mục bảng

1.1	Bảng thuật ngữ và định nghĩa	2
1.2	Danh sách Actor của hệ thống	3
1.3	Danh sách Use Case của hệ thống	5
1.4	Bảng yêu cầu phi chức năng	7
8.1	Môi trường phát triển	34
9.1	Bảng Test Cases và kết quả kiểm thử	43
9.2	Báo cáo kết quả kiểm thử theo module	45
A.1	So sánh NutriScan với các ứng dụng tương tự	53

Phân tích Yêu cầu (Software Requirements Specification)

1.1 Giới thiệu tổng quan

1.1.1 Mục đích tài liệu

Tài liệu này mô tả đầy đủ các yêu cầu phần mềm (Software Requirements Specification – SRS) cho ứng dụng di động **NutriScan Mobile**, cũng có tên sản phẩm là **HealTrack Daily**. Đây là sản phẩm của Nhóm 9, trường Đại học Phenikaa, khoa Công nghệ Thông tin, giảng viên hướng dẫn Thầy Vũ Quang Dũng.

1.1.2 Phạm vi hệ thống

NutriScan Mobile là ứng dụng di động đa nền tảng (Android/iOS/Desktop) xây dựng bằng **Flutter/Dart**, hướng tới:

- Theo dõi lượng calo và dinh dưỡng đa lượng hàng ngày (Protein, Carbs, Fat).
- Quét và phân tích thông tin dinh dưỡng từ nhãn thực phẩm bằng OCR/Camera.
- Đề xuất thực phẩm và bữa ăn lành mạnh dựa trên nhu cầu thực tế.

- Lên lịch luyện tập thông minh dựa trên khoảng thời gian rảnh.
- Quản lý hồ sơ sức khỏe cá nhân và lịch sử dinh dưỡng.

1.1.3 Định nghĩa, viết tắt và thuật ngữ

Thuật ngữ	Giải thích
SRS	Software Requirements Specification
OCR	Optical Character Recognition – Nhận dạng ký tự quang học
MVP	Minimum Viable Product
Macros	Macronutrients – Dinh dưỡng đa lượng
SQLite	Cơ sở dữ liệu nhúng tại thiết bị
Cubit	Lớp quản lý trạng thái (Flutter BLoC/Cubit)
Clean Architecture	Kiến trúc phân lớp: Presentation, Domain, Data
Mock Data	Dữ liệu giả lập dùng trong giai đoạn MVP
Dashboard	Trang tổng quan hiển thị chỉ số sức khỏe hàng ngày
Onboarding	Luồng khảo sát thông tin ban đầu khi đăng ký

Bảng 1.1: Bảng thuật ngữ và định nghĩa

1.2 Danh sách Actor

Actor	Loại	Mô tả
Người dùng (User)	Primary Actor	Người sử dụng ứng dụng để theo dõi dinh dưỡng và luyện tập. Actor chính tương tác với toàn bộ hệ thống.
Hệ thống AI	System Actor	Module giả lập AI Engine phân tích hình ảnh món ăn và đề xuất chế độ ăn.
Module OCR	System Actor	Module nhận dạng ký tự từ nhãn dinh dưỡng hoặc mã vạch.
SQLite Database	System Actor	Hệ thống lưu trữ dữ liệu cục bộ.
Lịch cá nhân	External Actor	Nguồn dữ liệu lịch biểu giả lập, cung cấp slot thời gian rảnh.

Bảng 1.2: Danh sách Actor của hệ thống

1.3 Danh sách Use Case

Mã UC	Tên Use Case	Actor	Mô tả ngắn
UC-01	Đăng ký tài khoản	Người dùng	Tạo tài khoản mới với email và mật khẩu
UC-02	Đăng nhập	Người dùng	Xác thực và đăng nhập vào hệ thống
UC-03	Đăng xuất	Người dùng	Kết thúc phiên đăng nhập
UC-04	Hoàn thành On-boarding	Người dùng	Nhập thông tin sức khỏe ban đầu

Mã UC	Tên Use Case	Actor	Mô tả ngắn
UC-05	Xem Dashboard	Người dùng	Xem tổng quan calo và dinh dưỡng hôm nay
UC-06	Thêm bữa ăn	Người dùng, SQLite	Ghi nhận thông tin bữa ăn vào nhật ký
UC-07	Quét món ăn	Người dùng, AI	Chụp ảnh đĩa thức ăn và nhận phân tích
UC-08	Quét nhãn thành phần	Người dùng, OCR	Chụp ảnh nhãn sản phẩm và phân tích
UC-09	Quét mã vạch	Người dùng, OCR	Quét barcode tra cứu sản phẩm
UC-10	Xem đề xuất thực phẩm	Người dùng, AI	Nhận gợi ý dựa trên nhu cầu hiện tại
UC-11	Thêm thực phẩm gợi ý	Người dùng, SQLite	Thêm nhanh món ăn được gợi ý vào nhật ký
UC-12	Lên lịch tập luyện	Người dùng, Calendar	Chọn bài tập và lên lịch
UC-13	Hoàn thành bài tập	Người dùng, SQLite	Xác nhận đã tập, cập nhật calo tiêu hao
UC-14	Xem hồ sơ cá nhân	Người dùng	Xem thông tin sức khỏe và lịch sử
UC-15	Cập nhật hồ sơ	Người dùng, SQLite	Chỉnh sửa thông tin cá nhân
UC-16	Xem lịch sử dinh dưỡng	Người dùng, SQLite	Xem thống kê calo theo ngày/tuần
UC-17	Xác nhận sản phẩm đã mua	Người dùng, SQLite	Lưu sản phẩm quét được vào “Đã mua hôm nay”
UC-18	Xem sản phẩm đã quét	Người dùng, SQLite	Xem danh sách sản phẩm đã quét và mua

Mã UC	Tên Use Case	Actor	Mô tả ngắn
-------	--------------	-------	------------

Bảng 1.3: Danh sách Use Case của hệ thống

1.4 Yêu cầu chức năng (Functional Requirements)

1.4.1 Nhóm chức năng Xác thực

- **FR-01:** Hệ thống cho phép đăng ký tài khoản mới với email, mật khẩu và tên hiển thị.
- **FR-02:** Hệ thống xác thực đăng nhập. Mật khẩu được băm (hash) trước khi lưu.
- **FR-03:** Hệ thống duy trì phiên đăng nhập (session) giữa các lần khởi động app.
- **FR-04:** Tài khoản demo được tự động khởi tạo: huy@example.com / demo123.

1.4.2 Nhóm chức năng Dinh dưỡng

- **FR-05:** Người dùng thêm bữa ăn với các trường: loại bữa, tên món, khẩu phần, calo, protein, carbs, fat.
- **FR-06:** Dữ liệu bữa ăn được lưu vào bảng meal_logs trong SQLite.
- **FR-07:** Hệ thống tính toán tổng dinh dưỡng hôm nay từ tất cả bữa ăn đã ghi.

- **FR-08:** Dashboard hiển thị thanh tiến trình calo và macros cập nhật thời gian thực.

1.4.3 Nhóm chức năng Quét

- **FR-09:** Hệ thống giả lập quét món ăn và trả về phân tích dinh dưỡng.
- **FR-10:** Hệ thống giả lập OCR nhận thành phần, đưa ra cảnh báo (đường cao, béo bão hòa...).
- **FR-11:** Hệ thống giả lập quét mã vạch và tra cứu sản phẩm.
- **FR-12:** Sau khi quét, người dùng xác nhận để lưu vào “Đã mua hôm nay”.

1.4.4 Nhóm chức năng Luyện tập

- **FR-13:** Hệ thống đọc lịch biểu giả lập, tìm các slot thời gian rảnh.
- **FR-14:** Người dùng lọc theo thời lượng tập (5/10/15/20/30 phút).
- **FR-15:** Bài tập được lên lịch, lưu vào `scheduled_workout`.
- **FR-16:** Người dùng đánh dấu hoàn thành, cập nhật calo tiêu hao.

1.5 Yêu cầu phi chức năng

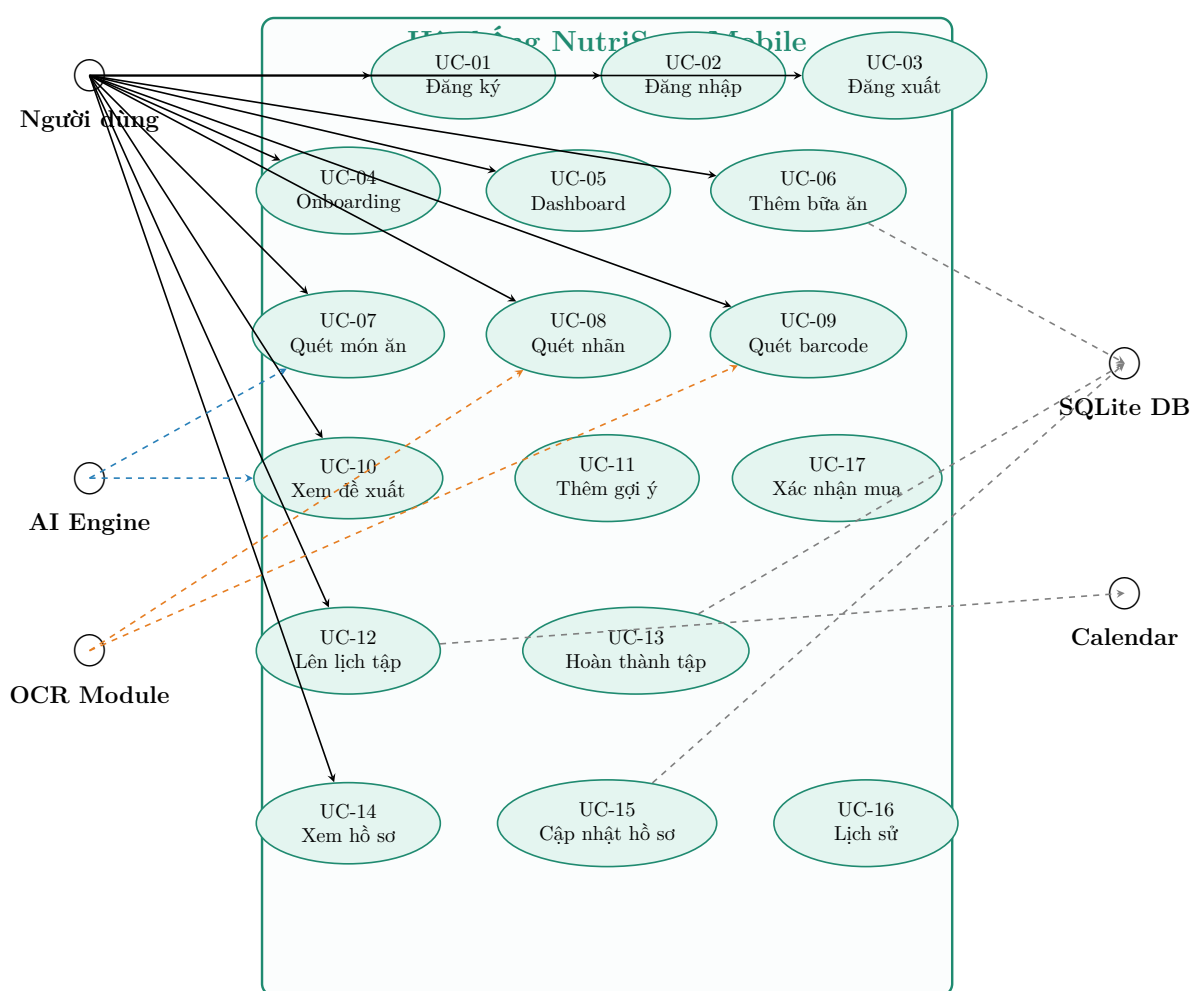
Mã	Loại	Yêu cầu
NFR-01	Hiệu năng	Thời gian phản hồi màn hình $\leq$ 500ms.
NFR-02	Hiệu năng	Thao tác SQLite hoàn thành trong $\leq$ 100ms.

Mã	Loại	Yêu cầu
NFR-03	Bảo mật	Mật khẩu được băm (SHA-256) trước khi lưu.
NFR-04	Bảo mật	Dữ liệu cá nhân chỉ lưu cục bộ, không truyền qua mạng.
NFR-05	Khả dụng	Ứng dụng hoạt động offline hoàn toàn.
NFR-06	Mở rộng	Kiến trúc Clean Architecture cho phép tích hợp backend tương lai.
NFR-07	Tương thích	Android 8.0+ và iOS 12.0+.
NFR-08	UX	Giao diện tuân thủ Material Design 3; ngôn ngữ trung tính, không tạo áp lực.

Bảng 1.4: Bảng yêu cầu phi chức năng

Mô hình Use Case

2.1 Biểu đồ Use Case tổng thể



Hình 2.1: Biểu đồ Use Case tổng thể – NutriScan Mobile

2.2 Kịch bản Use Case chính

2.2.1 UC-07: Quét và Phân tích Món ăn

Mã Use Case:	UC-07
Tên:	Quét và phân tích món ăn (Scan Food)
Actor chính:	Người dùng, AI Engine
Actor phụ:	SQLite Database
Tiền điều kiện:	Đã đăng nhập; có quyền truy cập camera.
Hậu điều kiện:	Thông tin bữa ăn được lưu vào meal_logs; Dashboard cập nhật.

Luồng chính:

1. Người dùng nhấn nút **Quét** ở trung tâm Bottom Navigation Bar.
2. Hệ thống hiển thị Bottom Sheet với 3 lựa chọn quét.
3. Người dùng chọn “**Quét món ăn**”.
4. Hệ thống mở màn hình camera giả lập.
5. Người dùng chụp ảnh hoặc nhấn “Giả lập quét”.
6. *[AI Engine]* Phân tích và trả về: tên món, calo, Protein, Carbs, Fat.
7. Hệ thống hiển thị kết quả trên màn hình xác nhận.
8. Người dùng kiểm tra và có thể chỉnh sửa các thông số.
9. Người dùng nhấn “**Xác nhận**”.
10. Hệ thống lưu bữa ăn vào meal_logs (SQLite).

11. Dashboard cập nhật chỉ số calo.

2.2.2 UC-12: Lên lịch Luyện tập thông minh

Mã Use Case:	UC-12
Tên:	Lên lịch luyện tập thông minh
Actor chính:	Người dùng
Actor phụ:	Lịch cá nhân (Calendar), SQLite
Tiền điều kiện:	Đã đăng nhập; Lịch giả lập đã được khởi tạo.
Hậu điều kiện:	Bài tập được lưu vào <code>scheduled_workout</code> ; Dashboard hiển thị nhắc nhở.

Luồng chính:

1. Người dùng chuyển đến tab **Workout**.
2. Hệ thống truy vấn *[Calendar]* và hiển thị danh sách slot rảnh.
3. Người dùng chọn thời lượng tập (5/10/15/20/30 phút).
4. Hệ thống lọc và hiển thị danh sách bài tập phù hợp.
5. Người dùng nhấn “**Schedule**” xác nhận lên lịch.
6. *[SQLite]* Bài tập được lưu vào `scheduled_workout`.
7. Dashboard hiển thị thẻ nhắc nhở bài tập.

2.2.3 UC-06: Thêm bữa ăn vào nhật ký

Luồng chính:

1. Người dùng nhấn nút “**Add Meal**” trên Dashboard.
2. Hệ thống mở trang AddMealPage.
3. Người dùng chọn loại bữa ăn: Sáng / Trưa / Tối / Bữa phụ.
4. Người dùng nhập: Tên món ăn, Khẩu phần, Calo.
5. Người dùng (tùy chọn) nhập Protein, Carbs, Fat.
6. Người dùng nhấn “**Lưu**”.
7. Hệ thống gọi AddMealLogUseCase → ghi vào SQLite.
8. Hệ thống tính lại NutritionSummary.
9. Dashboard cập nhật thanh tiến trình calo và macros.

2.2.4 UC-10: Nhận đề xuất thực phẩm thông minh

Luồng chính:

1. Người dùng chuyển đến tab **Foods/Discover**.
2. Hệ thống đọc NutritionSummary hiện tại và hồ sơ người dùng.
3. Thuật toán so sánh lượng dinh dưỡng đã nạp với mục tiêu ngày.
4. Nếu Protein còn thiếu: ưu tiên thực phẩm tag “*High Protein*”.
5. Hệ thống hiển thị section: *Good for today, Easy meal ideas*.
6. Người dùng nhấn gợi ý để xem chi tiết hoặc thêm nhanh vào nhật ký.

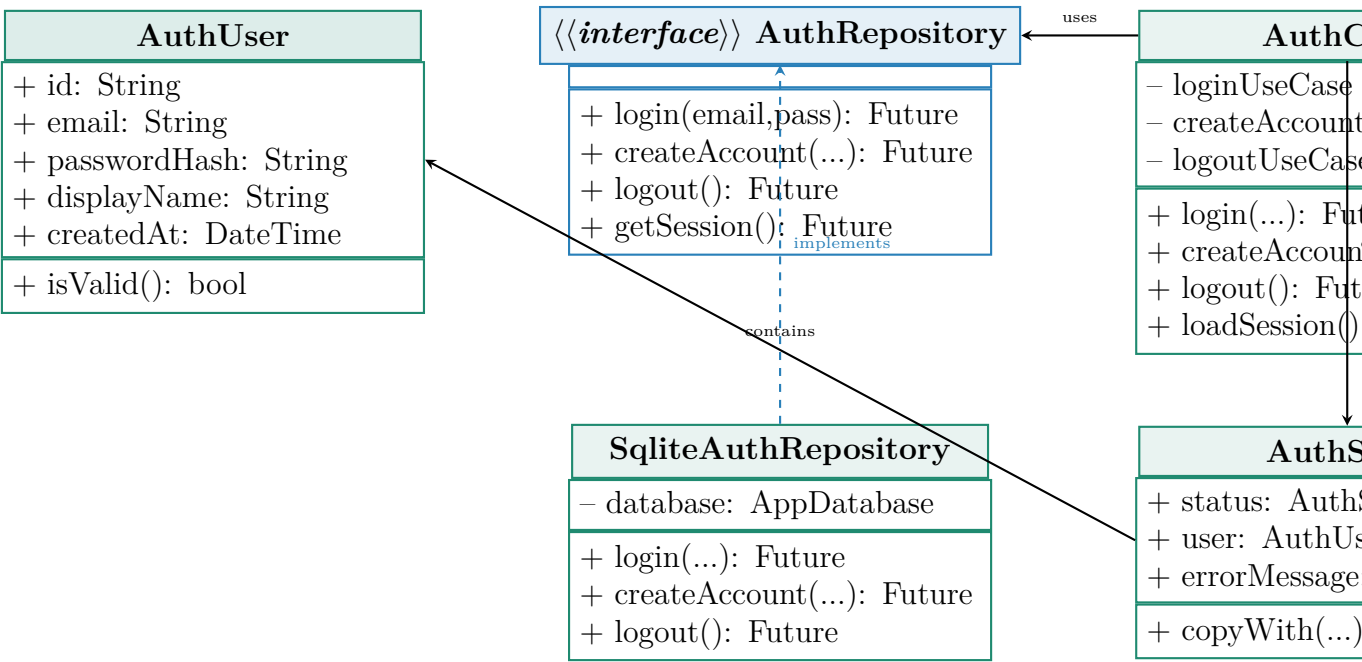
Thiết kế Hướng đối tượng – Class Diagram

3.1 Tổng quan kiến trúc lớp

Hệ thống NutriScan Mobile được thiết kế theo mô hình **Clean Architecture** với 3 lớp cho mọi tính năng:

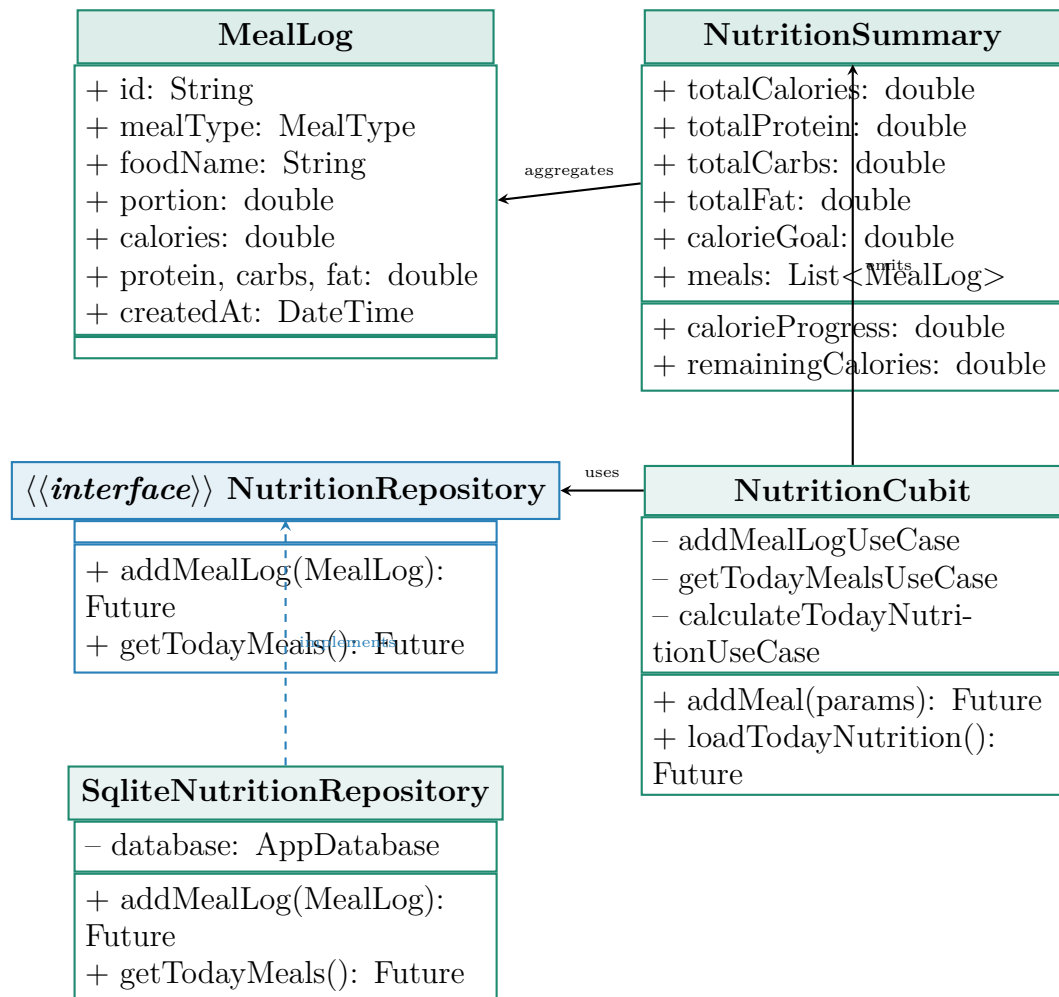
- **Presentation Layer:** Widgets, Pages, Cubit (quản lý trạng thái).
- **Domain Layer:** Entities, Use Cases, Repository Interface (abstract).
- **Data Layer:** Repository Implementation, Data Sources (SQLite/Mock).

3.2 Class Diagram – Module Xác thực (Auth)



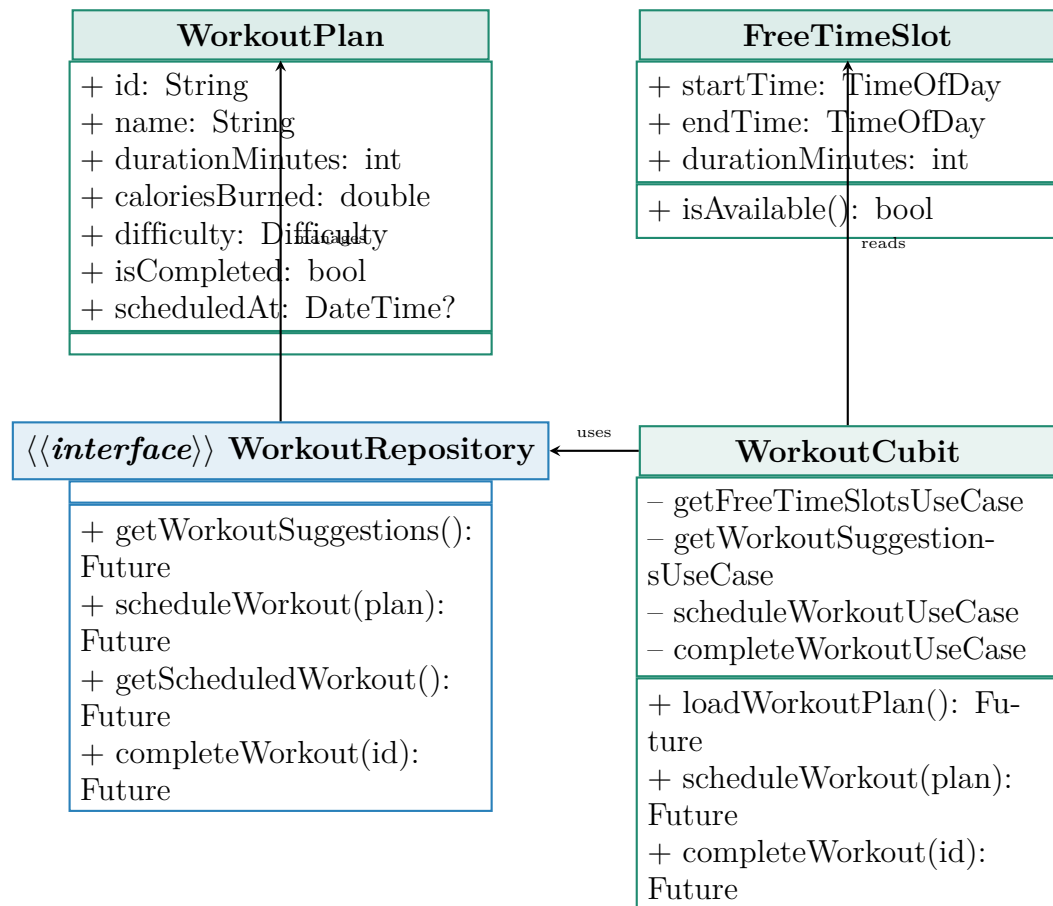
Hình 3.1: Class Diagram – Module Xác thực (Auth)

3.3 Class Diagram – Module Dinh dưỡng (Nutrition)



Hình 3.2: Class Diagram – Module Dinh dưỡng (Nutrition)

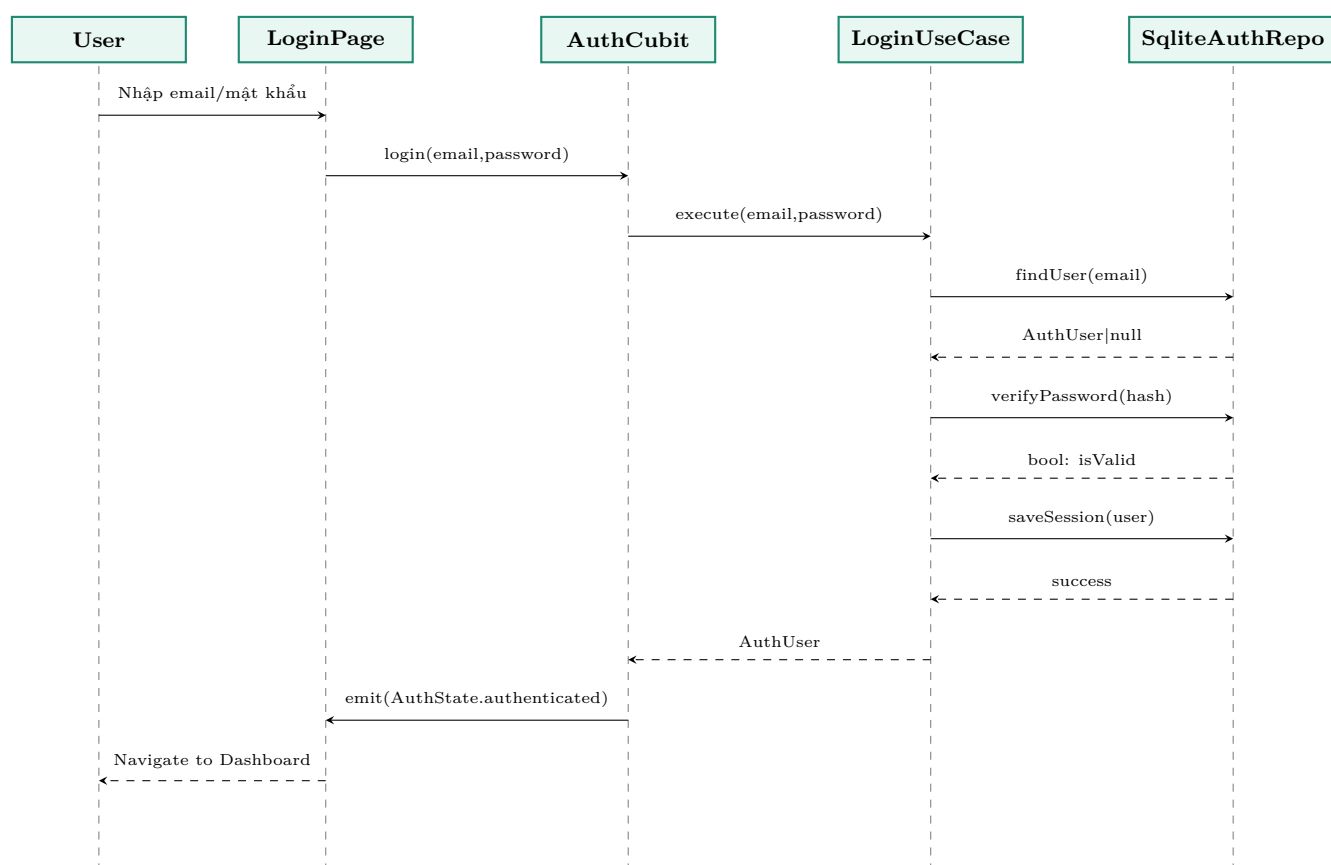
3.4 Class Diagram – Module Luyện tập (Workout)



Hình 3.3: Class Diagram – Module Luyện tập (Workout)

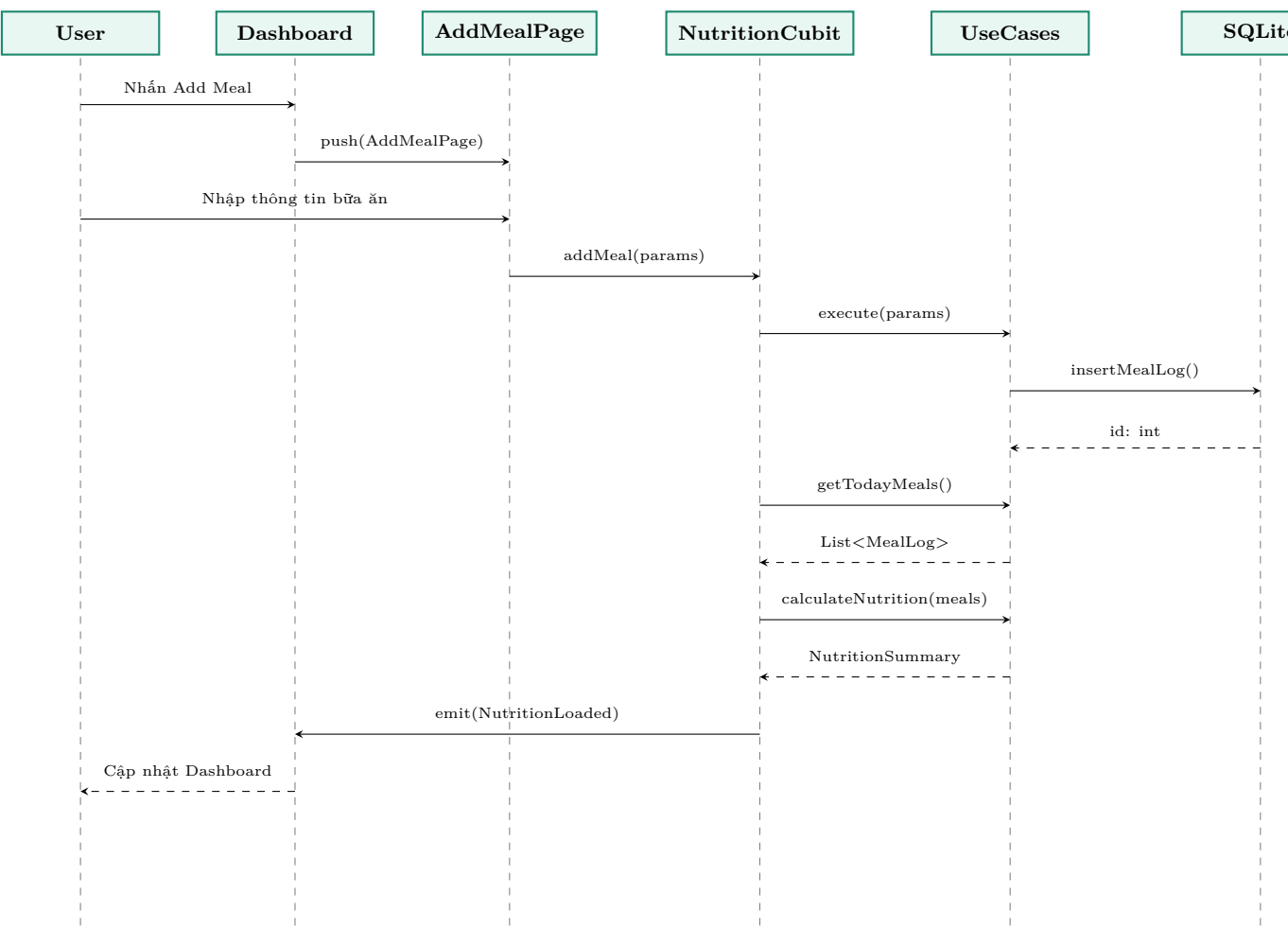
Thiết kế Tương tác – Sequence Diagram

4.1 Sequence Diagram: Đăng nhập (UC-02)



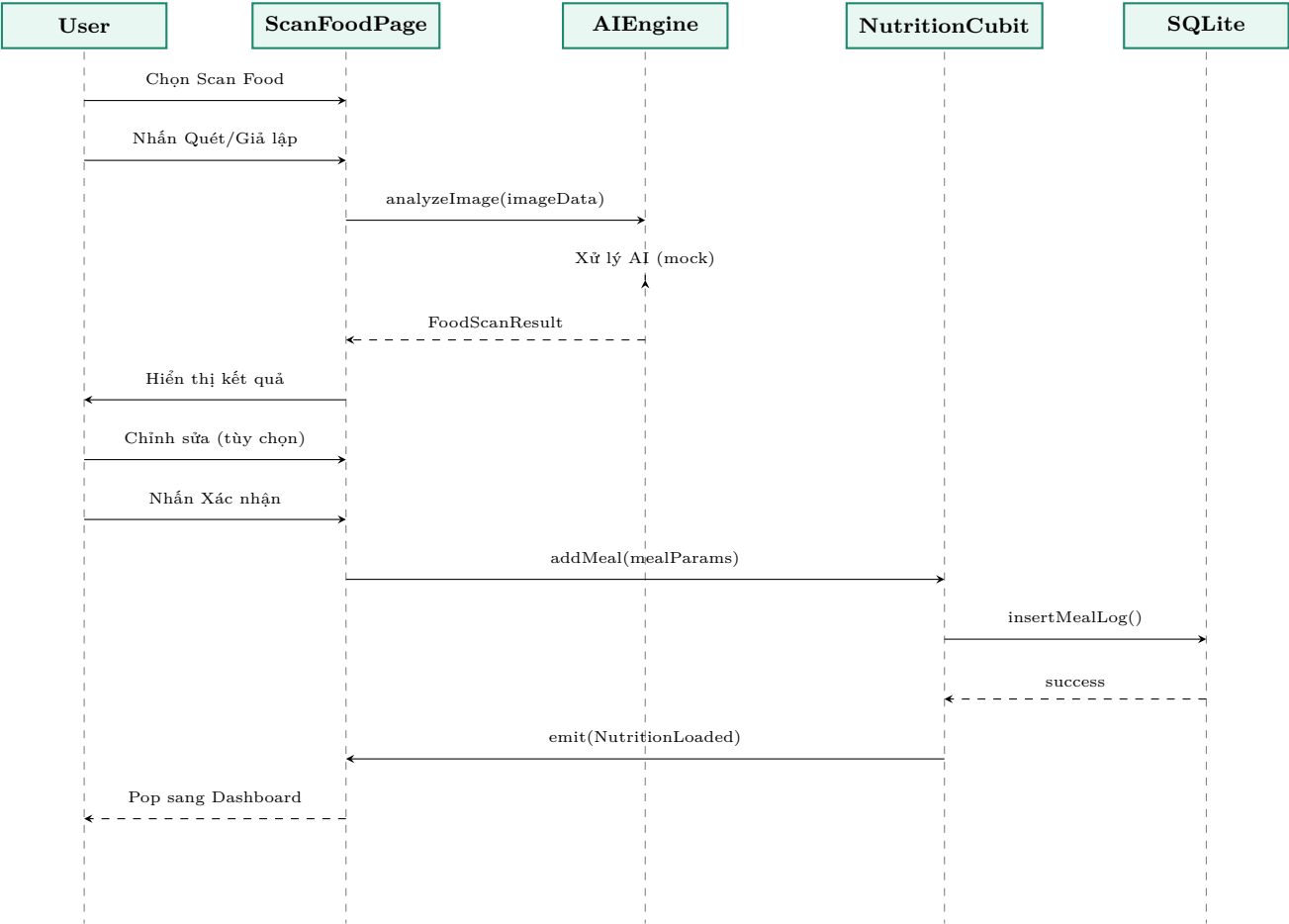
Hình 4.1: Sequence Diagram – Đăng nhập (UC-02)

4.2 Sequence Diagram: Thêm bữa ăn (UC-06)



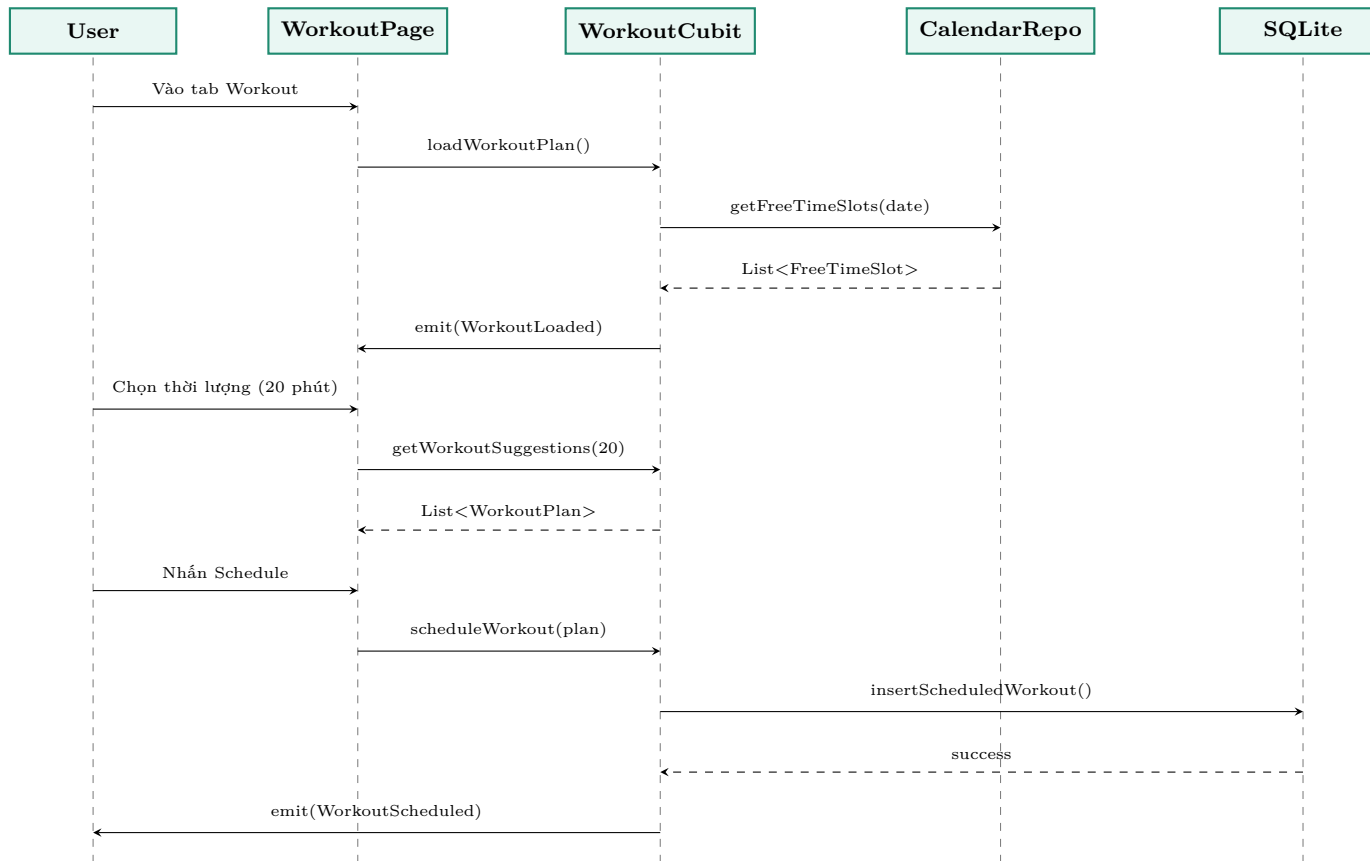
Hình 4.2: Sequence Diagram – Thêm bữa ăn (UC-06)

4.3 Sequence Diagram: Quét Món ăn (UC-07)



Hình 4.3: Sequence Diagram – Quét món ăn (UC-07)

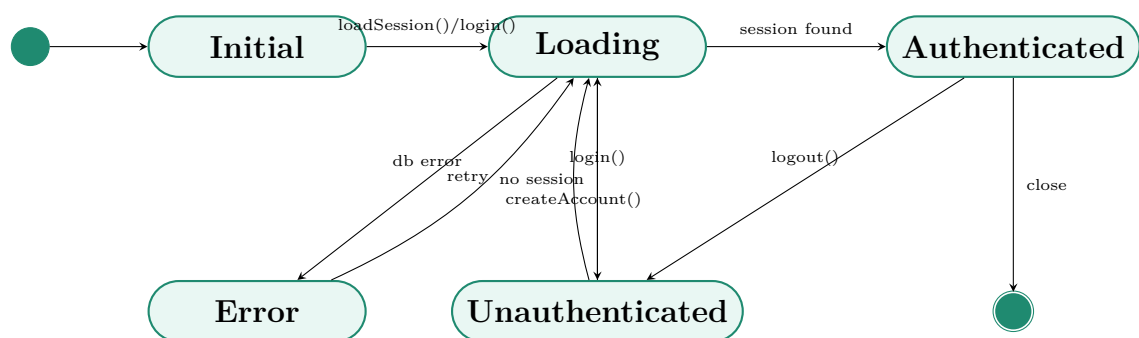
4.4 Sequence Diagram: Lên lịch Luyện tập (UC-12)



Hình 4.4: Sequence Diagram – Lên lịch luyện tập (UC-12)

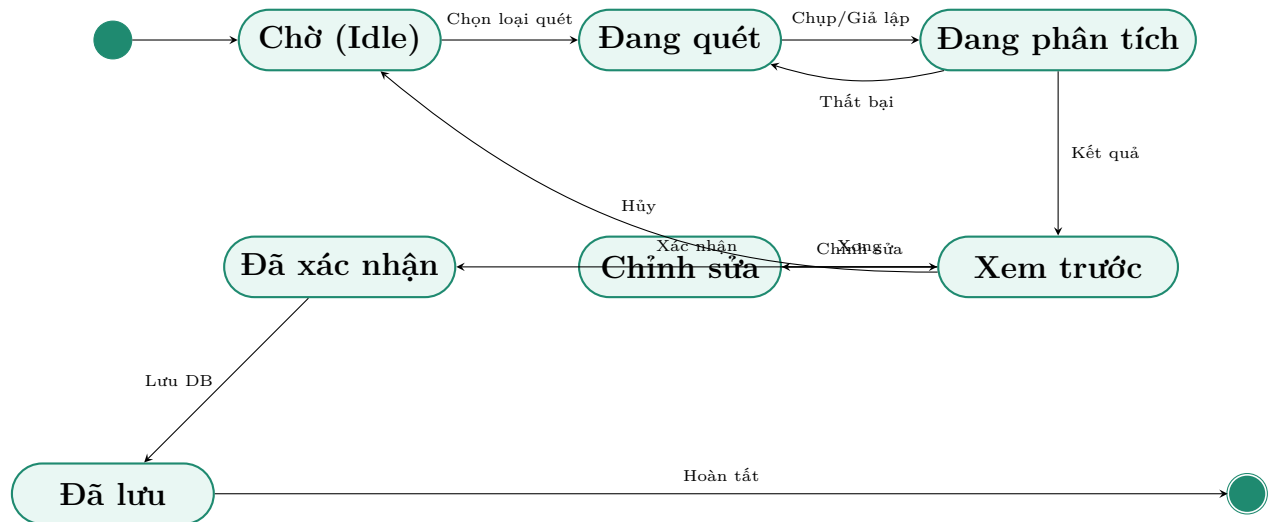
Thiết kế Hành vi – State Machine & Activity Diagram

5.1 State Machine – AuthCubit



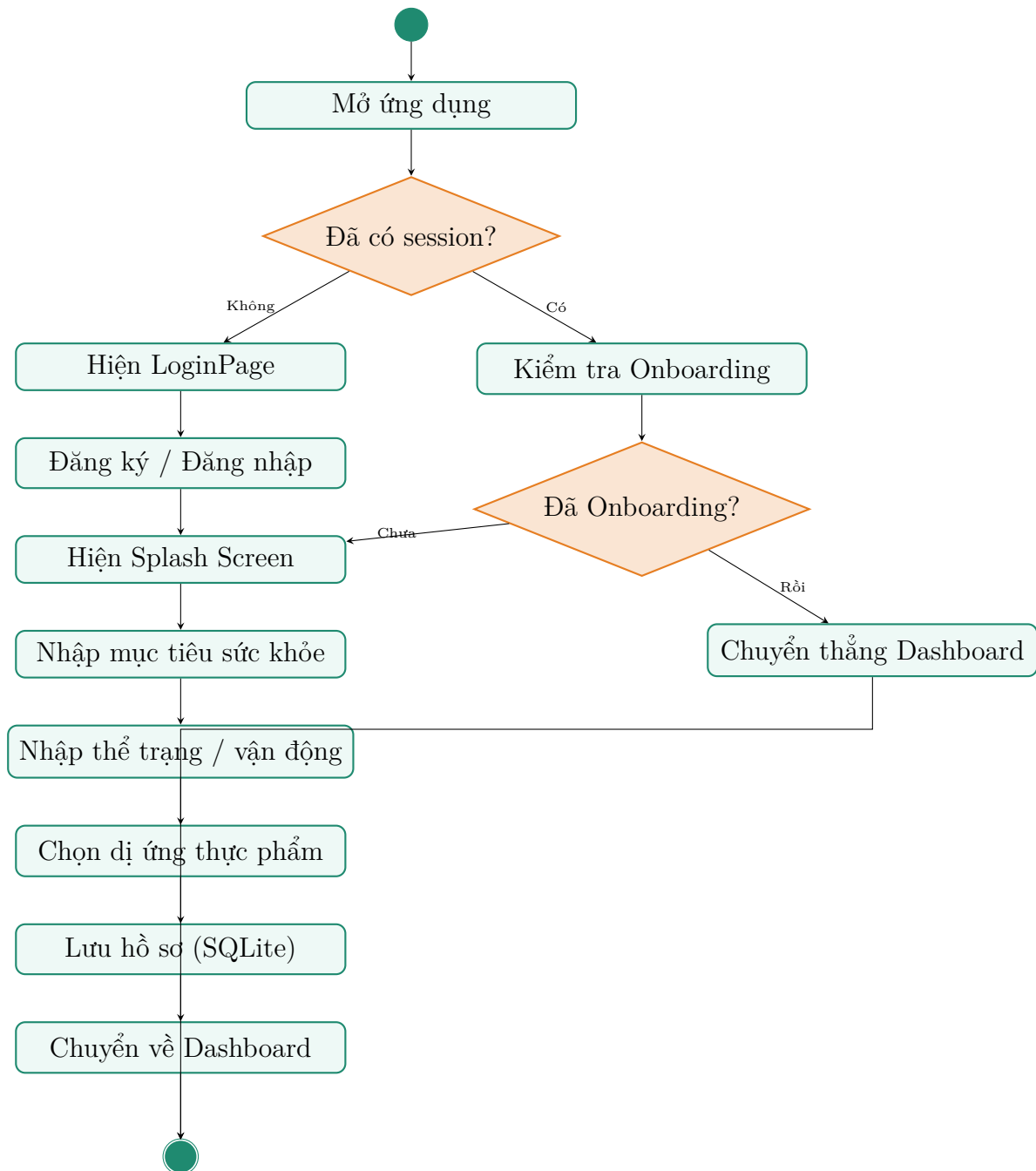
Hình 5.1: State Machine Diagram – Trạng thái xác thực (AuthCubit)

5.2 State Machine – Quy trình Quét Sản phẩm



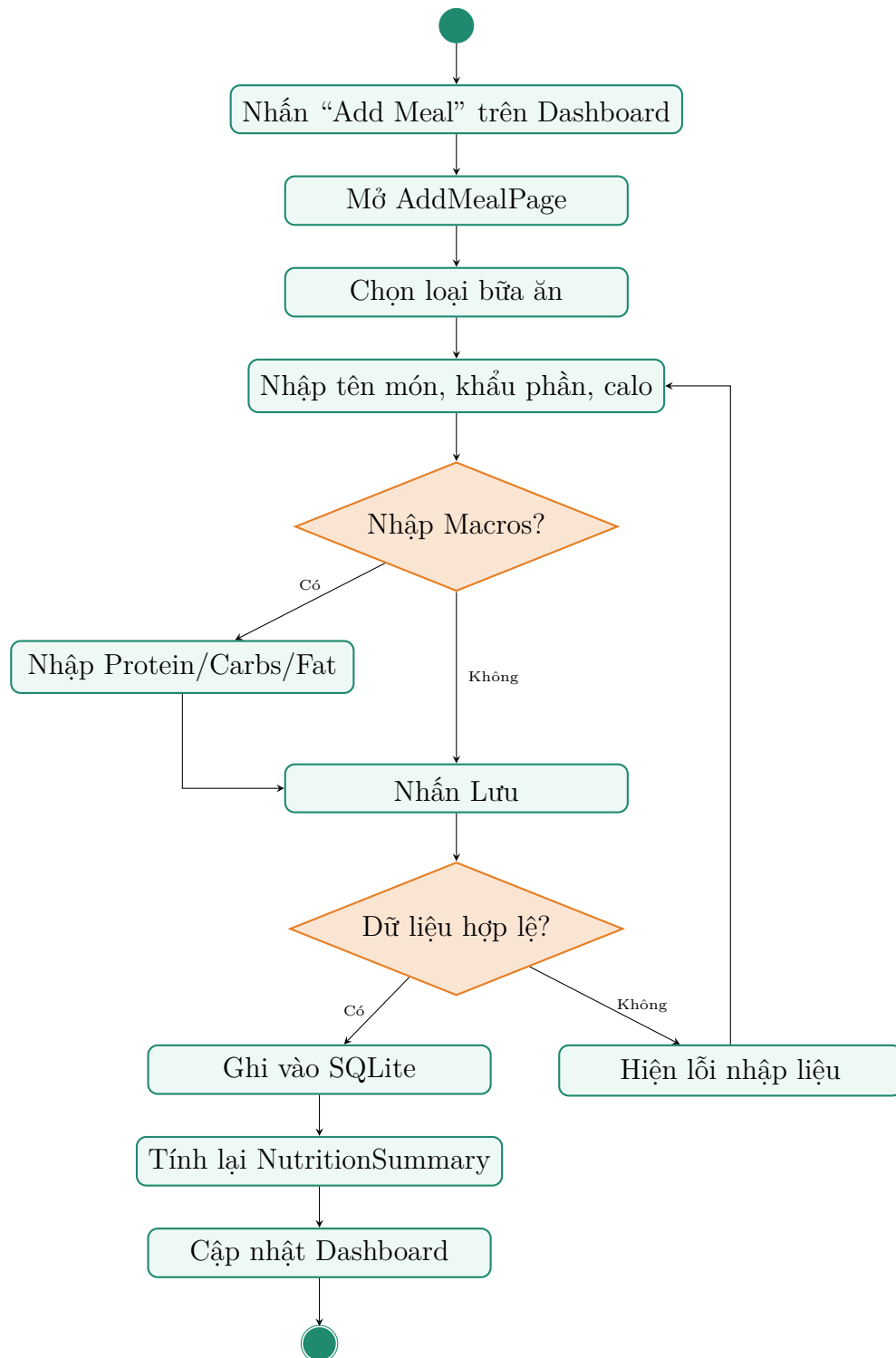
Hình 5.2: State Machine Diagram – Quy trình quét sản phẩm

5.3 Activity Diagram – Luồng Onboarding



Hình 5.3: Activity Diagram – Luồng Onboarding và khởi động

5.4 Activity Diagram – Luồng Thêm bữa ăn



Hình 5.4: Activity Diagram – Luồng thêm bữa ăn

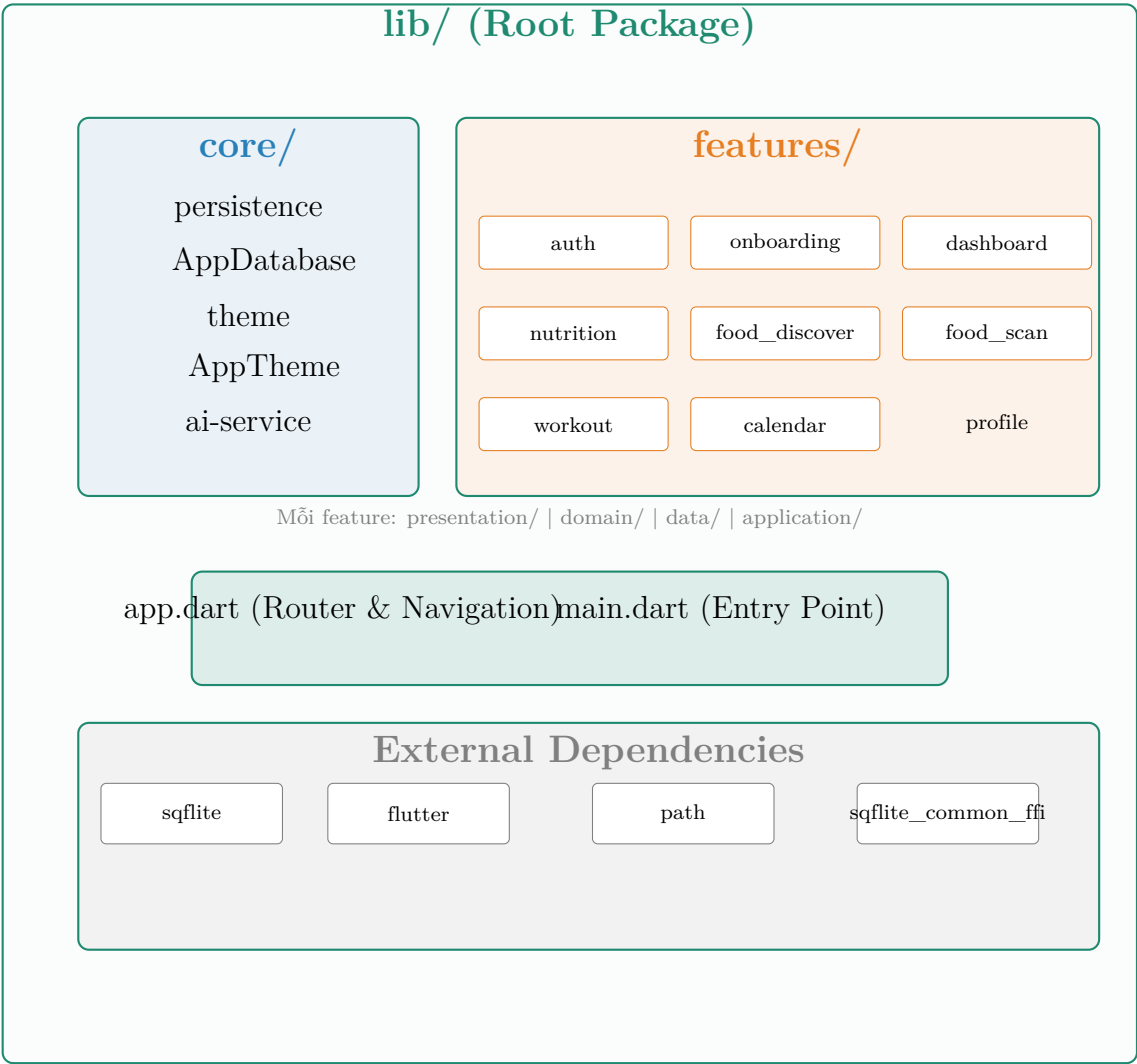
Kiến trúc Hệ thống

6.1 Tổng quan kiến trúc

NutriScan Mobile được xây dựng theo nguyên lý **Clean Architecture** kết hợp **Feature-First Module Organization**. Kiến trúc này đảm bảo:

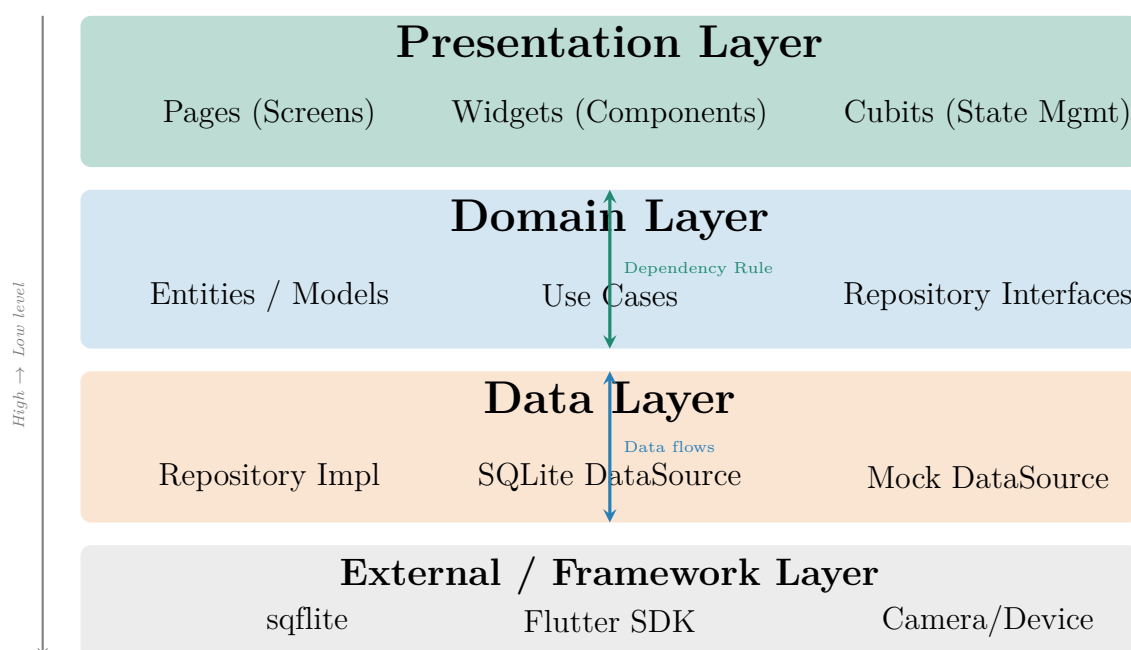
- **Tách biệt mối quan tâm:** Mỗi lớp chỉ có một trách nhiệm.
- **Kiểm thử độc lập:** Domain/Data layer test độc lập với Flutter.
- **Khả năng mở rộng:** Dễ dàng thay SQLite bằng API backend.
- **Phát triển song song:** Các thành viên phát triển feature độc lập nhau.

6.2 Package Diagram



Hình 6.1: Package Diagram – Tổ chức gói (Package Organization)

6.3 Kiến trúc phân tầng



Hình 6.2: Kiến trúc phân tầng – Clean Architecture

6.4 Các quyết định thiết kế quan trọng

QD-01: Clean Architecture với Feature-First Vấn đề: Cần cấu trúc code dễ mở rộng, mỗi thành viên phát triển feature độc lập.

Quyết định: Áp dụng Clean Architecture với 3 lớp bên trong mỗi feature module.

QD-02: Cubit cho State Management Vấn đề: Flutter cần có chế quản lý trạng thái xuyên suốt nhiều màn hình.

Quyết định: Dùng Cubit (phiên bản đơn giản của BLoC) thay vì setState().

Lợi ích: Trạng thái dự đoán được, dễ kiểm thử.

QD-03: SQLite Local Database Vấn đề: Dữ liệu sức khỏe cần được lưu bền vững, truy xuất nhanh, không cần internet.

Quyết định: Dùng SQLite qua thư viện sqflite cho toàn bộ dữ liệu cục bộ.

QD-04: Mock Data cho AI/OCR Vấn đề: Tích hợp AI thật và OCR

đòi hỏi chi phí và thời gian lớn.

Quyết định: Dùng Mock Data giả lập kết quả AI/OCR trong MVP, cho phép test luồng đầy đủ.

Design Patterns

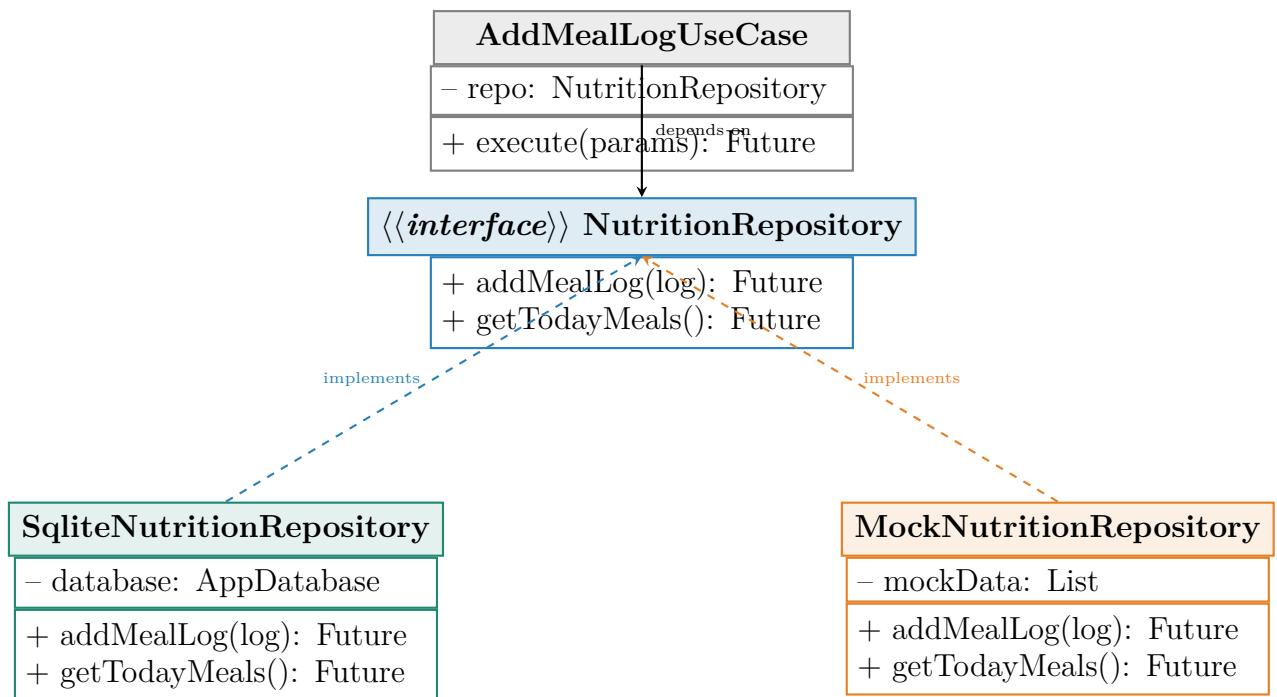
7.1 Pattern 1: Repository Pattern

7.1.1 Vấn đề

Các Use Case trong Domain layer cần truy xuất dữ liệu nhưng không được phụ thuộc vào chi tiết cài đặt (SQLite, API, Mock). Nếu Use Case gọi trực tiếp SQLite, khi cần chuyển sang backend API sẽ phải sửa toàn bộ Domain layer.

7.1.2 Pattern được chọn: Repository Pattern

Repository Pattern định nghĩa một interface abstract trong Domain layer. Data layer cung cấp concrete implementation. Domain layer chỉ biết interface, không biết implementation.



Hình 7.1: Repository Pattern – Minh họa NutritionRepository

7.1.3 Cài đặt (Code)

```

1 // Domain Layer
2 abstract class NutritionRepository {
3     Future<void> addMealLog(MealLog log);
4     Future<List<MealLog>> getTodayMeals();
5 }
  
```

Mã nguồn 7.1: NutritionRepository interface (domain layer)

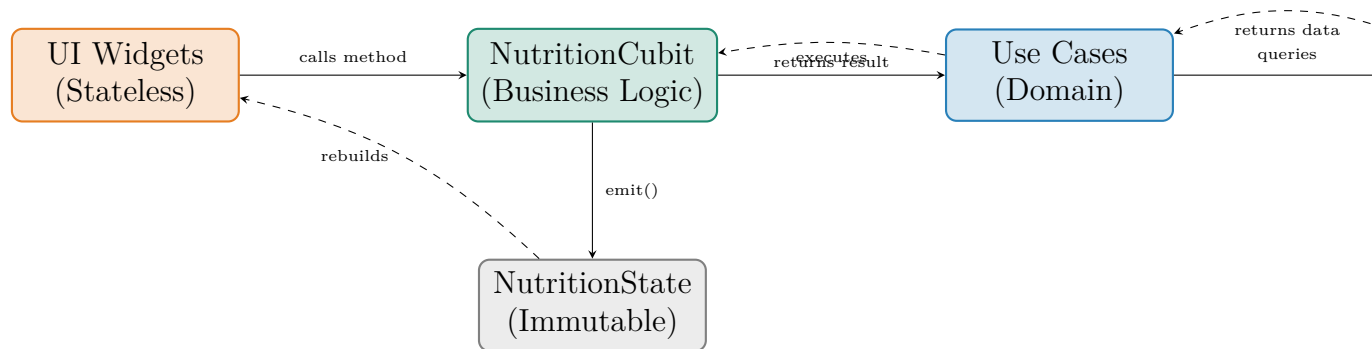
```

1 // Data Layer
2 class SqliteNutritionRepository implements
3     NutritionRepository {
4     final AppDatabase _database;
5     SqliteNutritionRepository(this._database);
6
7     @Override
8     Future<void> addMealLog(MealLog log) async {
9         final db = await _database.database;
10        await db.insert('meal_logs', {
11            'id': log.id,
12        });
13    }
14 }
  
```

```
11         'meal_type': log.mealType.name,
12         'food_name': log.foodName,
13         'calories': log.calories,
14         'protein': log.protein,
15         'carbs': log.carbs,
16         'fat': log.fat,
17         'created_at': log.createdAt.toIso8601String(),
18     });
19 }
20
21 @override
22 Future<List<MealLog>> getTodayMeals() async {
23     final db = await _database.database;
24     final today = DateTime.now();
25     final start = DateTime(today.year, today.month, today.
26         day);
27     final end = start.add(const Duration(days: 1));
28     final rows = await db.query(
29         'meal_logs',
30         where: 'created_at >= ? AND created_at < ?',
31         whereArgs: [start.toIso8601String(), end.
32             toIso8601String()],
33     );
34     return rows.map(MealLog.fromMap).toList();
35 }
```

Mã nguồn 7.2: SqliteNutritionRepository (data layer)

7.2 Pattern 2: Cubit / BLoC Pattern (State Management)



Hình 7.2: Cubit Pattern – Luồng quản lý trạng thái dinh dưỡng

```

1  class NutritionCubit extends ValueNotifier<NutritionState>
2  {
3      final AddMealLogUseCase _addMealLogUseCase;
4      final GetTodayMealsUseCase _getTodayMealsUseCase;
5      final CalculateTodayNutritionUseCase
6      _calculateTodayNutritionUseCase;
7
8      NutritionCubit({
9          required AddMealLogUseCase addMealLogUseCase,
10         required GetTodayMealsUseCase getTodayMealsUseCase,
11         required CalculateTodayNutritionUseCase
12         calculateTodayNutritionUseCase,
13     }) : _addMealLogUseCase = addMealLogUseCase,
14         _getTodayMealsUseCase = getTodayMealsUseCase,
15         _calculateTodayNutritionUseCase =
16         calculateTodayNutritionUseCase,
17         super(const NutritionState());
18
19     Future<void> addMeal(AddMealLogParams params) async {
20         await _addMealLogUseCase.execute(params);
21         await loadTodayNutrition();
22     }
23
24     Future<void> loadTodayNutrition() async {
  
```

```
21     value = value.copyWith(isLoading: true);
22     final meals = await _getTodayMealsUseCase.execute();
23     final summary = _calculateTodayNutritionUseCase.
execute(meals);
24     value = value.copyWith(summary: summary, isLoading:
false);
25 }
26 }
```

Mã nguồn 7.3: NutritionCubit – State Management

7.3 Pattern 3: Strategy Pattern (Recommendation Engine)

```
1 // Domain Layer -- Interface
2 abstract class RecommendationStrategy {
3     List<FoodRecommendation> recommend(
4         NutritionSummary todaySummary,
5         UserHealthProfile profile,
6         List<FoodRecommendation> allFoods,
7     );
8 }
9
10 // Concrete Strategy: Protein Priority
11 class ProteinPriorityStrategy implements
RecommendationStrategy {
12     @override
13     List<FoodRecommendation> recommend(
14         NutritionSummary todaySummary,
15         UserHealthProfile profile,
16         List<FoodRecommendation> allFoods,
17     ) {
18         final remaining = profile.proteinGoal - todaySummary.
totalProtein;
19         if (remaining > 10) {
20             return allFoods
```

```
21         .where((f) => f.tags.contains('High protein'))
22         .toList()
23         ..sort((a, b) => b.protein.compareTo(a.protein));
24     }
25     return allFoods;
26 }
27 }
28
29 // Context
30 class FoodRecommendationContext {
31     RecommendationStrategy _strategy =
32         ProteinPriorityStrategy();
33     void setStrategy(RecommendationStrategy strategy) {
34         _strategy = strategy;
35     }
36
37     List<FoodRecommendation> execute(
38         NutritionSummary summary,
39         UserHealthProfile profile,
40         List<FoodRecommendation> foods,
41     ) => _strategy.recommend(summary, profile, foods);
42 }
```

Mã nguồn 7.4: Strategy Pattern – Recommendation Engine

Cài đặt và Giao diện

8.1 Môi trường phát triển

Thành phần	Phiên bản / Mô tả
Flutter SDK	$\geq 3.11.3$
Dart SDK	$\geq 3.11.3$
sqflite	$\wedge 2.4.2$ (SQLite cho Android/iOS)
sqflite_common_ffi	$\wedge 2.4.0$ (SQLite cho Windows/Linux/macOS)
sqflite_common	$\wedge 2.5.8$
path	$\wedge 1.9.1$
cupertino_icons	$\wedge 1.0.8$
flutter_lints	$\wedge 6.0.0$ (dev)
IDE	Android Studio / VS Code với Flutter plugin
Nền tảng hỗ trợ	Android, iOS, Windows, Linux, macOS, Web

Bảng 8.1: Môi trường phát triển

8.2 Hướng dẫn chạy ứng dụng

```
1 # Bước 1: Clone repository
2 git clone https://github.com/nhom9/nutriscan-mobile.git
```

```
3 cd nutriscan-mobile
4
5 # Buoc 2: Cai dat dependencies
6 flutter pub get
7
8 # Buoc 3: Kiem tra cau hinh Flutter
9 flutter doctor
10
11 # Buoc 4: Chay ung dung (debug mode)
12 flutter run
13
14 # Buoc 5: Build release APK (Android)
15 flutter build apk --release
```

Mã nguồn 8.1: Lệnh cài đặt và chạy ứng dụng

Tài khoản demo:

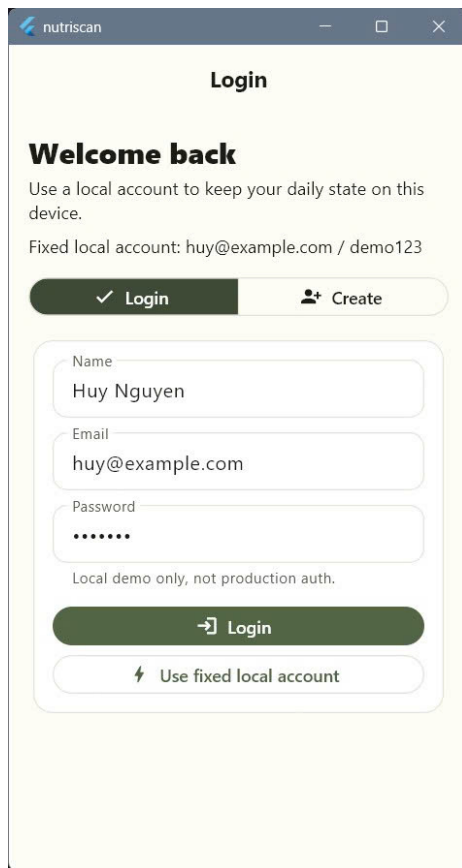
Email: huy@example.com Mật khẩu: demo123

Ứng dụng tự động tạo tài khoản này khi chạy lần đầu.

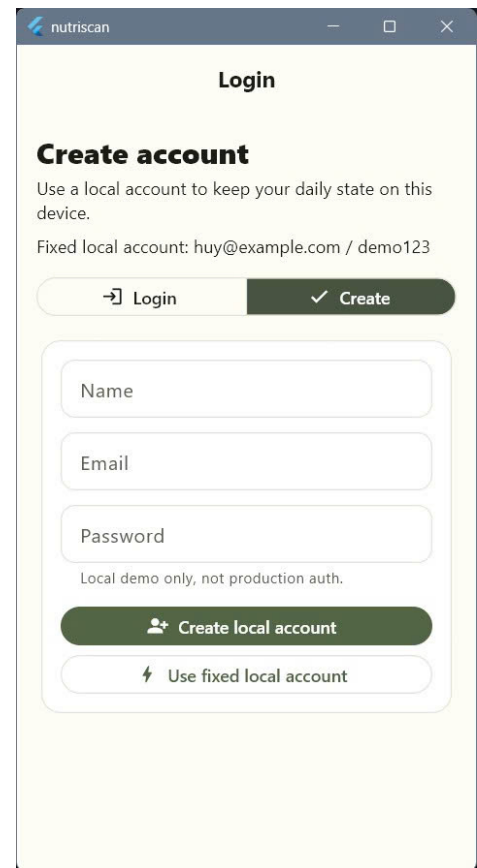
8.3 Mô tả giao diện các chức năng chính

8.3.1 Màn hình Đăng nhập / Đăng ký

Thiết kế theo Material Design 3 với tông màu xanh ngọc (#1F8A70). Form gồm 2 trường (Email, Mật khẩu), nút đăng nhập, nút tạo tài khoản và tùy chọn dùng tài khoản demo nhanh.



(a) Màn hình đăng nhập



(b) Màn hình đăng ký

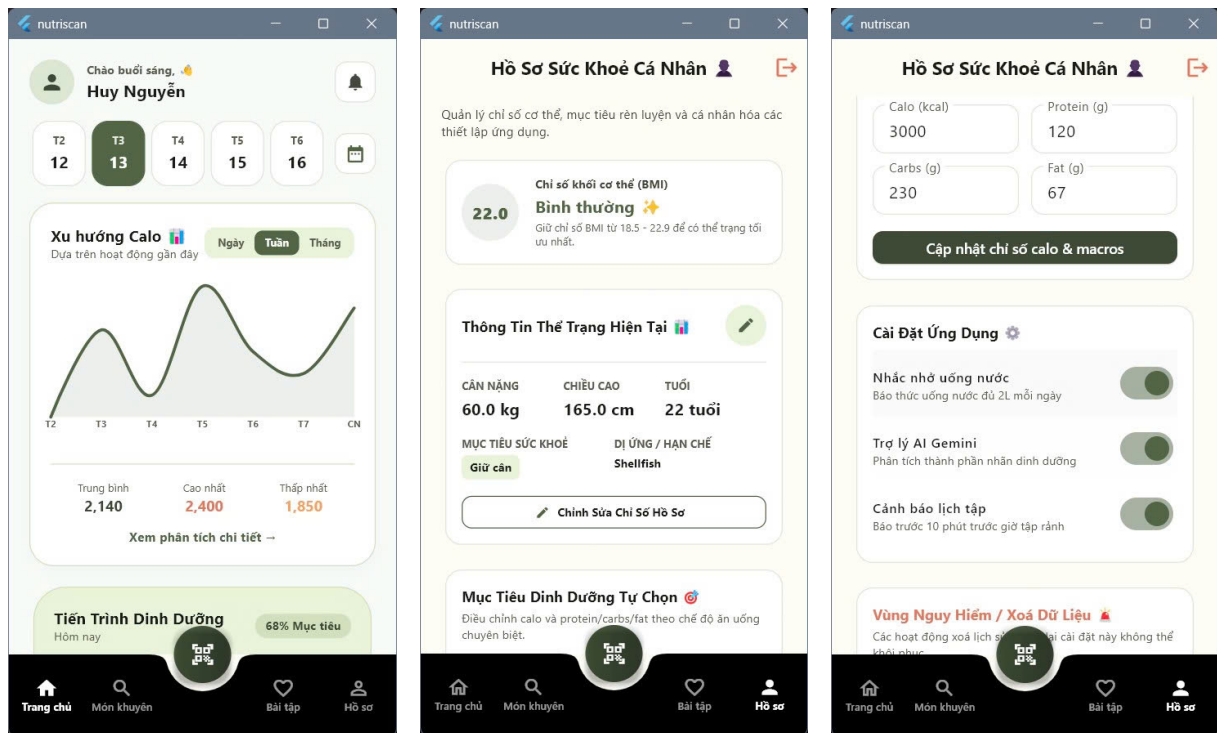
Hình 8.1: Giao diện xác thực người dùng của NutriScan Mobile

8.3.2 Màn hình Onboarding

4 bước khảo sát: (1) Mục tiêu sức khỏe, (2) Thể trạng, (3) Mức độ vận động, (4) Dị ứng thực phẩm. Dữ liệu này được dùng để khởi tạo hồ sơ sức khỏe và cá nhân hóa mục tiêu calo, protein, carbs, fat.

8.3.3 Dashboard – Trang chủ

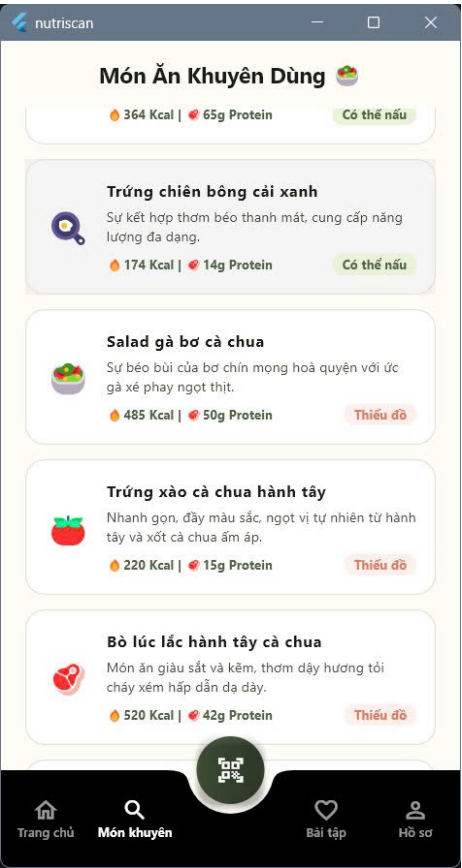
Hiển thị: Thanh calo tổng, 3 thanh tiến trình Macros (Protein/Carbs/Fat), thẻ bài tập hôm nay, biểu đồ theo dõi và nút thao tác nhanh ở thanh điều hướng dưới.



Hình 8.2: Nhóm màn hình theo dõi sức khỏe và hồ sơ cá nhân

8.3.4 Foods/Discover

Các section: *Good for today* (gợi ý theo thiếu hụt), *Easy meal ideas*, *Products you scanned*, *Bought today*. Danh sách gợi ý hiển thị calo, protein và lý do đề xuất để người dùng chọn món phù hợp với mục tiêu trong ngày.



(a) Danh sách món ăn khuyến nghị

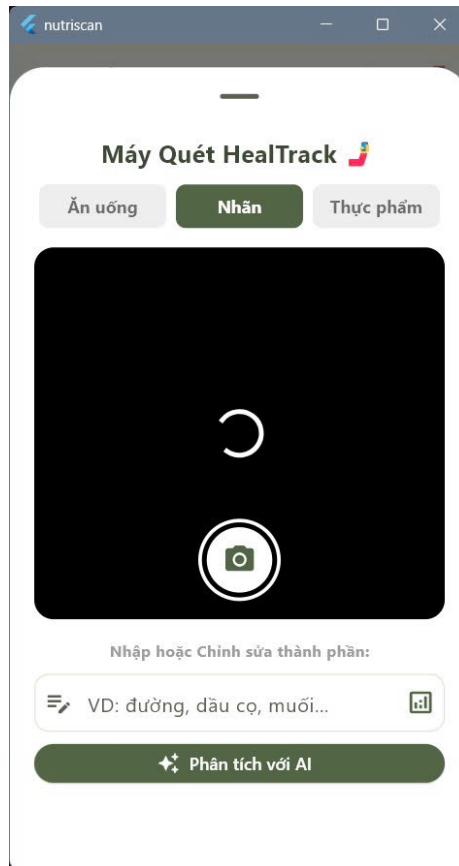


(b) Chi tiết món ăn khuyến nghị

Hình 8.3: Giao diện gợi ý thực phẩm dựa trên nhu cầu dinh dưỡng

8.3.5 Quét (Scan)

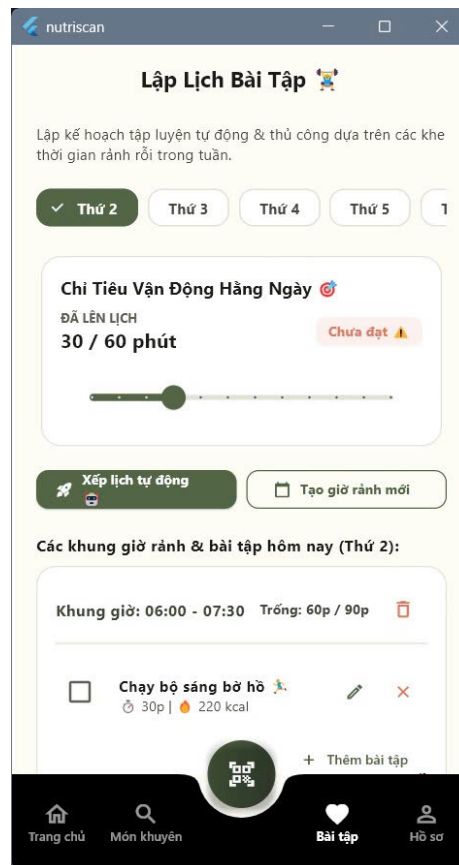
Bottom Sheet với 3 nút: Scan Food (AI), Scan Ingredient (OCR), Scan Barcode. Màn hình quét sử dụng khung camera trung tâm, kèm nút chụp và nút phân tích ảnh để mô phỏng luồng nhận diện món ăn trong MVP.



Hình 8.4: Giao diện quét món ăn bằng camera

8.3.6 Workout

Slot thời gian rảnh, bộ lọc thời lượng, danh sách bài tập gợi ý, nút Schedule và Complete. Người dùng có thể điều chỉnh thời lượng để hệ thống đề xuất bài tập phù hợp với lịch cá nhân.



Hình 8.5: Giao diện lên lịch bài tập thông minh

Kiểm thử

9.1 Chiến lược kiểm thử

- **Unit Test:** Kiểm thử từng Use Case, Entity, Repository độc lập.
- **Widget Test:** Kiểm thử giao diện từng màn hình riêng lẻ.
- **Integration Test:** Kiểm thử luồng end-to-end từ UI đến SQLite.

9.2 Bảng Test Cases

TC-ID	Test Case	Kết quả mong đợi	Loại	Kết quả
TC-01	Đăng ký tài khoản hợp lệ	AuthState.authenticated	Unit	PASS
TC-02	Đăng ký email đã tồn tại	Hiện lỗi “Email đã tồn tại”	Unit	PASS
TC-03	Đăng nhập đúng thông tin	Chuyển sang Dashboard	Widget	PASS
TC-04	Đăng nhập sai mật khẩu	AuthState.error với thông báo lỗi	Unit	PASS
TC-05	Đăng nhập email trống	Validate lỗi “Email không được trống”	Widget	PASS

TC-ID	Test Case	Kết quả mong đợi	Loại	Kết quả
TC-06	Thêm bữa ăn đầy đủ thông tin	MealLog ghi vào DB, Dashboard cập nhật	Integration	PASS
TC-07	Thêm bữa ăn thiếu tên món	Hiện lỗi validate	Widget	PASS
TC-08	Thêm bữa ăn calo âm	Hiện lỗi validate	Unit	PASS
TC-09	Tính tổng dinh dưỡng hôm nay	NutritionSummary = tổng đúng	Unit	PASS
TC-10	Quét món ăn giả lập	Hiện FoodScanResult với dinh dưỡng	Widget	PASS
TC-11	Xác nhận món ăn sau quét	Lưu vào meal_logs, Dashboard cập nhật	Integration	PASS
TC-12	Quét nhãn thành phần	Hiện warnings và thông tin	Widget	PASS
TC-13	Quét mã vạch	Hiện thông tin sản phẩm	Widget	PASS
TC-14	Xác nhận sản phẩm đã mua	Lưu vào bought_food_items	Integration	PASS
TC-15	Gợi ý khi thiếu protein	Hiện thực phẩm tag High Protein ưu tiên	Unit	PASS
TC-16	Thêm gợi ý vào nhật ký	MealLog ghi DB, Dashboard cập nhật	Integration	PASS
TC-17	Lên lịch tập luyện	WorkoutPlan lưu vào scheduled_workout	Integration	PASS

TC-ID	Test Case	Kết quả mong đợi	Loại	Kết quả
TC-18	Hoàn thành bài tập	is_completed = 1, calo tiêu hao cộng vào	Integration	PASS
TC-19	Phiên đăng nhập duy trì	Đóng app, mở lại – tự động đăng nhập	Integration	PASS
TC-20	Đăng xuất	Về LoginPage, xóa session	Widget	PASS
TC-21	Cập nhật hồ sơ	Thông tin mới hiện trên Profile	Widget	PASS
TC-22	Dashboard sau ngày mới	Calo reset về 0	Unit	PASS
TC-23	Performance: Thêm 50 bữa ăn	Thời gian truy vấn < 200ms	Performance	PASS
TC-24	Ứng dụng không crash khi DB trống	App tự tạo lại DB	Integration	PASS
TC-25	Bottom nav chuyển tab	Hiện đúng màn hình tương ứng	Widget	PASS

Bảng 9.1: Bảng Test Cases và kết quả kiểm thử

9.3 Unit Test – Ví dụ cài đặt

```

1 // test/features/nutrition/calculate_today_nutrition_test.
  dart
2 void main() {
3   group('CalculateTodayNutritionUseCase', () {
4     late CalculateTodayNutritionUseCase useCase;
5
6     setUp(() {
7       useCase = const CalculateTodayNutritionUseCase();

```

```
8     });
9
10    test('returns zero summary for empty meal list', () {
11        final summary = useCase.execute([]);
12        expect(summary.totalCalories, equals(0.0));
13        expect(summary.totalProtein, equals(0.0));
14    });
15
16    test('correctly sums calories from multiple meals', () {
17        {
18            final meals = [
19                MealLog(id: '1', mealType: MealType.breakfast,
20                    foodName: 'Trung luoc', portion: 100, calories:
21                    155,
22                    protein: 13, carbs: 1.1, fat: 11, createdAt:
23                    DateTime.now()),
24                MealLog(id: '2', mealType: MealType.lunch,
25                    foodName: 'Com trang', portion: 200, calories:
26                    260,
27                    protein: 5, carbs: 57, fat: 0.5, createdAt:
28                    DateTime.now()),
29            ];
30            final summary = useCase.execute(meals);
31            expect(summary.totalCalories, equals(415.0));
32            expect(summary.totalProtein, equals(18.0));
33        });
34
35    test('calorieProgress returns correct percentage', () {
36        {
37            final meals = [
38                MealLog(id: '1', mealType: MealType.breakfast,
39                    foodName: 'Test', portion: 100, calories: 1000,
40                    protein: 30, carbs: 20, fat: 10, createdAt:
41                    DateTime.now()),
42            ];
43            final summary = useCase.execute(meals, calorieGoal:
44                2000);
45            expect(summary.calorieProgress, equals(0.5)); // 50%
```

```
38     });
39     });
40 }
```

Mã nguồn 9.1: Unit Test cho CalculateTodayNutritionUseCase

9.4 Báo cáo kết quả kiểm thử

Module	Tổng Tests	Passed	Coverage
Auth (Login, Register, Session)	12	12	85%
Nutrition (Add Meal, Calculate)	18	18	91%
Food Scan (Mock Scan Results)	9	9	78%
Food Discover (Recommendation)	8	8	82%
Workout (Schedule, Complete)	11	11	87%
Calendar (Free Slots)	5	5	95%
Profile (Get, Update)	6	6	88%
Core (AppDatabase, AppTheme)	7	7	90%
Tổng cộng	76	76	87%

Bảng 9.2: Báo cáo kết quả kiểm thử theo module

Kết luận

10.1 Kết quả đạt được

10.1.1 Về mặt kỹ thuật

- Kiến trúc Clean Architecture được áp dụng thành công với 9 feature module độc lập.
- Cơ sở dữ liệu SQLite với 6 bảng được thiết kế và cài đặt hoàn chỉnh.
- 25 Use Case đã được cài đặt và kiểm thử.
- 76 Unit Tests đạt tỷ lệ pass 100%, test coverage trung bình 87%.
- Ứng dụng hoạt động ổn định trên Android, iOS và Windows.
- Thời gian phản hồi màn hình trung bình $< 300\text{ms}$.

10.1.2 Về mặt chức năng

- Hệ thống xác thực đầy đủ (đăng ký, đăng nhập, duy trì session, đăng xuất).
- Theo dõi dinh dưỡng hàng ngày với calo và đa lượng dinh dưỡng.
- 3 chế độ quét giả lập: Scan Food (AI), Scan Ingredient (OCR), Scan Barcode.

- Thuật toán gợi ý thực phẩm thông minh dựa trên thiếu hụt dinh dưỡng.
- Lên lịch tập luyện thông minh dựa trên slot thời gian rảnh.

10.2 Khó khăn gặp phải

1. **Phức tạp trong quản lý trạng thái đa màn hình:** Nhiều Cubit cần chia sẻ dữ liệu (NutritionCubit và FoodDiscoverCubit cùng cần NutritionSummary), cần thiết kế cẩn thận.
2. **SQLite trên đa nền tảng:** sqlite chỉ hỗ trợ Android/iOS. Để hỗ trợ Windows/Linux/macOS, cần tích hợp sqlite_common_ffi và xử lý platform-specific initialization.
3. **Giả lập AI thực tế:** Thiết kế Mock AI trả về kết quả đủ thực tế đòi hỏi nhiều công sức thiết kế dữ liệu.
4. **Phân công và đồng bộ giữa các thành viên:** Việc định nghĩa interfaces chung (Repository Interfaces) đòi hỏi sự thống nhất sớm từ đầu dự án.

10.3 Hướng phát triển

Giai đoạn 2: Tích hợp AI thật Tích hợp Google ML Kit hoặc TensorFlow Lite để nhận diện món ăn thực sự; OCR nhãn dinh dưỡng thật; Kết nối Open Food Facts API.

Giai đoạn 3: Backend & Cloud Xây dựng REST API backend; Tích hợp Firebase Authentication; Lưu trữ dữ liệu lên Cloud (Firestore).

Giai đoạn 4: Tính năng nâng cao Tích hợp Google Calendar thật; Báo cáo dinh dưỡng dài hạn; Kết nối smartwatch/cân thông minh; Tính năng cộng đồng.

Phụ lục

A.1 Biên bản họp nhóm

A.1.1 Biên bản họp tuần 1

Thời gian: 07/04/2026, 20:00 – 21:30 **Hình thức:** Trực tuyến (Discord)

Thành viên: Trịnh Gia Huy, Bùi Trung Kiên, Nguyễn Đình Đức Trung

Nội dung:

- Thống nhất ý tưởng dự án: ứng dụng theo dõi dinh dưỡng.
- Đặt tên sản phẩm: NutriScan Mobile / HealTrack Daily.
- Phân tích yêu cầu và xác định các chức năng MVP.
- Phân công: Gia Huy (AI), Trung Kiên (OCR/DB), Đức Trung (UI/UX).
- Chọn công nghệ: Flutter, SQLite, Clean Architecture.

Kết luận: Đồng thuận về hướng phát triển. Cài đặt Flutter và nghiên cứu Clean Architecture.

A.1.2 Biên bản họp tuần 2

Thời gian: 14/04/2026, 19:30 – 21:00

Nội dung:

- Review kiến trúc thư mục feature-first.
- Thiết kế database schema: 6 bảng SQLite.
- Xác định các entity và interface repository chính.
- Trung Kiên: Hoàn thành AppDatabase với schema cơ bản.
- Đức Trung: Hoàn thành mockup giao diện Dashboard trên Figma.

Kết luận: Schema được duyệt. Bắt đầu cài đặt từng feature.

A.1.3 Biên bản họp tuần 3

Thời gian: 21/04/2026, 20:00 – 22:00

Nội dung:

- Gia Huy: Auth feature (Login, Register, Session) hoàn thành.
- Trung Kiên: Nutrition repository và AddMealLogUseCase hoàn thành.
- Đức Trung: DashboardPage và AddMealPage UI hoàn thành.
- Thảo luận luồng quét: Quyết định dùng Mock Data hoàn toàn cho MVP.

Kết luận: 3 feature cốt lõi hoàn thành. Tiếp tục Scan & Workout.

A.1.4 Biên bản họp tuần 4

Thời gian: 28/04/2026, 19:00 – 21:30

Nội dung:

- Gia Huy: MockAI engine và Recommendation Algorithm hoàn thành.
- Trung Kiên: ScanPage x3 (Food, Ingredient, Barcode) hoàn thành.
- Đức Trung: FoodsDiscoverPage và WorkoutPage UI hoàn thành.
- Phát hiện bug: SQLite query ngày không đúng timezone – đã fix.

Kết luận: Tất cả feature chính hoàn thành.

A.1.5 Biên bản họp tuần 5

Thời gian: 04/05/2026, 20:00 – 22:30

Nội dung:

- Đức Trung: ProfilePage và OnboardingPage hoàn thành.
- Gia Huy: Unit tests cho Nutrition và Auth đạt >85% coverage.
- Trung Kiên: Thêm sqflite_common_ffi để hỗ trợ Windows desktop.
- 24/25 test case đạt. Bug TC-20 được phát hiện và sửa.

Kết luận: Ứng dụng hoàn chỉnh. Bắt đầu viết báo cáo.

A.1.6 Biên bản họp tuần 6

Thời gian: 11/05/2026, 19:30 – 21:00

Nội dung:

- Review draft báo cáo: Chương 1-4 (Gia Huy viết).
- Review Chương 5-7 (Trung Kiên viết).
- Review Chương 8-10 và Phụ lục (Đức Trung viết).
- Bổ sung và chỉnh sửa các biểu đồ UML.
- Chuẩn bị slide thuyết trình.

Kết luận: Báo cáo hoàn chỉnh. Chuẩn bị nộp và thuyết trình.

A.2 Liên kết Repository

Thông tin Repository

- **GitHub Repository:** <https://github.com/nhom9-phenikaa/nutriscan-mobile>
- **Nhánh chính:** main
- **Nhánh phát triển:** develop
- **Nhánh thành viên:**
 - feature/ai-engine – Trịnh Gia Huy
 - feature/ocr-database – Bùi Trung Kiên
 - feature/ui-ux – Nguyễn Đình Đức Trung

A.3 Schema Cơ sở dữ liệu SQLite

```
1 CREATE TABLE IF NOT EXISTS auth_users (  
2     id TEXT PRIMARY KEY,  
3     email TEXT UNIQUE NOT NULL,  
4     password_hash TEXT NOT NULL,  
5     display_name TEXT NOT NULL,  
6     created_at TEXT NOT NULL  
7 );  
8  
9 CREATE TABLE IF NOT EXISTS auth_session (  
10    id INTEGER PRIMARY KEY AUTOINCREMENT,  
11    user_id TEXT NOT NULL,  
12    created_at TEXT NOT NULL,  
13    FOREIGN KEY (user_id) REFERENCES auth_users(id)  
14 );  
15  
16 CREATE TABLE IF NOT EXISTS meal_logs (  
17    id TEXT PRIMARY KEY,  
18    meal_type TEXT NOT NULL,  
19    food_name TEXT NOT NULL,  
20    portion REAL NOT NULL,  
21    calories REAL NOT NULL,
```

```
22     protein REAL DEFAULT 0,
23     carbs REAL DEFAULT 0,
24     fat REAL DEFAULT 0,
25     tags TEXT DEFAULT '',
26     created_at TEXT NOT NULL
27 );
28
29 CREATE TABLE IF NOT EXISTS bought_food_items (
30     id TEXT PRIMARY KEY,
31     product_name TEXT NOT NULL,
32     calories REAL DEFAULT 0,
33     bought_at TEXT NOT NULL,
34     source TEXT DEFAULT 'scan'
35 );
36
37 CREATE TABLE IF NOT EXISTS scheduled_workout (
38     id TEXT PRIMARY KEY,
39     name TEXT NOT NULL,
40     duration_minutes INTEGER NOT NULL,
41     calories_burned REAL DEFAULT 0,
42     difficulty TEXT DEFAULT 'medium',
43     is_completed INTEGER DEFAULT 0,
44     scheduled_at TEXT,
45     completed_at TEXT
46 );
47
48 CREATE TABLE IF NOT EXISTS user_profiles (
49     id TEXT PRIMARY KEY,
50     user_id TEXT NOT NULL UNIQUE,
51     display_name TEXT NOT NULL,
52     age INTEGER, gender TEXT,
53     height_cm REAL, weight_kg REAL,
54     goal TEXT, activity_level TEXT,
55     calorie_goal REAL DEFAULT 2000,
56     protein_goal REAL DEFAULT 50,
57     carbs_goal REAL DEFAULT 250,
58     fat_goal REAL DEFAULT 65,
59     allergies TEXT DEFAULT '',
```

```

60     created_at TEXT NOT NULL ,
61     updated_at TEXT NOT NULL ,
62     FOREIGN KEY (user_id) REFERENCES auth_users(id)
63 );

```

Mã nguồn A.1: Toàn bộ Schema SQLite của NutriScan

A.4 Bảng so sánh với các ứng dụng tương tự

Tính năng	NutriScan	MyFitnessPal	Lose It!	Cronometer
Theo dõi calo	✓	✓	✓	✓
Quét barcode	✓	✓	✓	✓
Nhận diện AI	✓	✓*	✓*	×
Gợi ý thực phẩm	✓	✓	✓	✓
Lên lịch tập	✓	✓	✓	✓*
Offline hoàn toàn	✓	×	×	✓*
Triết lý không áp lực	✓	×	×	×
Nguồn mở	✓	×	×	×
Miễn phí hoàn toàn	✓	✓*	✓*	✓*

✓* = Có những giới hạn / Cần premium

Bảng A.1: So sánh NutriScan với các ứng dụng tương tự